



In [2]: !pip install biopython

```
Collecting biopython
  Downloading biopython-1.87-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (13 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from biopython) (2.0.2)
Downloading biopython-1.87-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (3.2 MB)
----- 3.2/3.2 MB 31.0 MB/s eta 0:00:00
Installing collected packages: biopython
Successfully installed biopython-1.87
```

```
In [3]: import os
import gzip
from Bio import SeqIO

# --- CONFIGURATION ---
input_filename = "uniprot_sprot.fasta.gz" # Matches the file you uploaded
output_filename = "hp_dataset_2laks.txt"
target_limit = 200000

# HP Mapping (Biochemical Hydrophobicity)
hp_map = {
    'A':'H', 'V':'H', 'L':'H', 'I':'H', 'P':'H', 'F':'H', 'W':'H', 'M':'H',
    'G':'P', 'S':'P', 'T':'P', 'C':'P', 'Y':'P', 'N':'P', 'Q':'P',
    'D':'P', 'E':'P', 'K':'P', 'R':'P', 'H':'P'
}

def to_hp(seq):
    return "".join([hp_map.get(aa, 'P') for aa in seq])

# --- 1. INITIAL DATASET REPORT ---
print("🔍 Analyzing uploaded UniProt file...")
raw_size = os.path.getsize(input_filename) / (1024 * 1024)
print(f"File Name: {input_filename}")
print(f"Compressed Size: {raw_size:.2f} MB")

# --- 2. TRANSFORMATION & COUNTING ---
print("\n🔄 Starting Transformation (Amino Acid -> HP)...")
processed_count = 0

with gzip.open(input_filename, "rt") as handle, open(output_filename, "w") as out:
    for record in SeqIO.parse(handle, "fasta"):
        # We filter for length 20-60 to keep RL training efficient
        if 20 <= len(record.seq) <= 60:
            hp_seq = to_hp(str(record.seq))
            out.write(hp_seq + "\n")
            processed_count += 1

        if processed_count >= target_limit:
            break

# --- 3. FINAL REPORT ---
```

```

final_size = os.path.getsize(output_filename) / (1024 * 1024)
print("\n✅ TRANSFORMATION COMPLETE")
print("-" * 30)
print(f"Total HP Sequences Count: {processed_count:,}")
print(f"Final Dataset Size: {final_size:.2f} MB")
print(f"Dataset Saved As: {output_filename}")
print("-" * 30)

```

🔍 Analyzing uploaded UniProt file...

File Name: uniprot_sprot.fasta.gz

Compressed Size: 89.13 MB

⚙ Starting Transformation (Amino Acid -> HP)...

✅ TRANSFORMATION COMPLETE

Total HP Sequences Count: 14,660

Final Dataset Size: 0.62 MB

Dataset Saved As: hp_dataset_2laks.txt

```

In [4]: import os
import gzip
from Bio import SeqIO

# --- SETTINGS ---
input_file = "uniprot_sprot.fasta.gz"
output_file = "hp_transformed_2laks.txt"
limit = 200000

# HP Mapping Rules
hp_map = {
    'A':'H', 'V':'H', 'L':'H', 'I':'H', 'P':'H', 'F':'H', 'W':'H', 'M':'H',
    'G':'P', 'S':'P', 'T':'P', 'C':'P', 'Y':'P', 'N':'P', 'Q':'P',
    'D':'P', 'E':'P', 'K':'P', 'R':'P', 'H':'P'
}

def to_hp(seq):
    return "".join([hp_map.get(aa, 'P') for aa in seq])

# 1. SCAN ORIGINAL DATASET
print("--- ORIGINAL DATASET REPORT ---")
total_raw = 0
test_sentences = []

with gzip.open(input_file, "rt") as handle:
    for record in SeqIO.parse(handle, "fasta"):
        if total_raw < 5:
            test_sentences.append(str(record.seq))
            total_raw += 1
        # Quick scan limit to avoid waiting too long for the raw count
        if total_raw >= 570000: break

print(f"Total Sequences found in original file: ~{total_raw:,}")

```

```

print("\nFirst 5 Test Sentences (Raw Amino Acids):")
for i, s in enumerate(test_sentences, 1):
    print(f"{i}. {s[:70]}...") # Showing first 70 chars for readability

# 2. PERFORM TRANSFORMATION
print("\n--- STARTING TRANSFORMATION ---")
processed_count = 0

with gzip.open(input_file, "rt") as handle, open(output_file, "w") as out:
    for record in SeqIO.parse(handle, "fasta"):
        # Filtering for length 20-60 (Standard for RL projects)
        if 20 <= len(record.seq) <= 60:
            hp_sequence = to_hp(str(record.seq))
            out.write(hp_sequence + "\n")
            processed_count += 1

        if processed_count >= limit:
            break

# 3. FINAL DATASET REPORT
print("\n--- TRANSFORMED DATASET REPORT ---")
print(f"Total HP Sequences Created: {processed_count:,}")
print(f"Columns: [Sequence (HP-encoded)]")
print(f"File Saved as: {output_file}")

print("\nFirst 5 Transformed Rows (Your New Dataset):")
with open(output_file, "r") as f:
    for _ in range(5):
        print(f.readline().strip())

```

--- ORIGINAL DATASET REPORT ---

Total Sequences found in original file: ~570,000

First 5 Test Sentences (Raw Amino Acids):

1. MAFSAEDVLKEYDRRRRMEALLLSLYPNDRKLLDYKEWSPPRVQVECPKAPVEWNNPPSEKGLIVGHFS...
2. MSIIGATRLQNDKSDTYSAGPCYAGGCSAFTPRGTCGKDWDLGEQTCASGFCTSQPLCARIKKTQVCGLR...
3. MASNTVSAQGGSNRPVRDFSNIQDVAQFLLFDPIWNEQPGSIVPWKMNREQALAERYPELQTSEPSSEDYS...
4. MYQAINPCPQSWYGSPQLEREIVCKMSGAPHYPNYYPVHPNALGGAWFDTSLNARSLTTTTPSLTTCTPPS...
5. MARPLLGKTSSVRRRLESLSACSIFFFLRKFCQKMASLVFLNSPVYQMSNILLTERRQVDRAMGGSDDDG...

--- STARTING TRANSFORMATION ---

--- TRANSFORMED DATASET REPORT ---

Total HP Sequences Created: 14,660

Columns: [Sequence (HP-encoded)]

File Saved as: hp_transformed_2laks.txt

First 5 Transformed Rows (Your New Dataset):

```

HPHPPPPPPPHPPHHHHHPHPHHHHHPHHHHPPPPPPPPPHPPPPPHPPHPHPHPPPPPP
HHHPHHPPPPPHPPHHHHHPHPHHHHHPHPHPPPPPHPPPPHPPPPHPPPPHPPHPHP
HPHPHHHPHPHPPHPHHHHHPHPHHHHHPHPHPPPPHPPHPHPHHHPH
HHHPHPHPHPPHPPPPPPHPPPPPPHPPHHPPHHHPHPHPPPHPPHHHPHPPPPPP
HPHPHPHHPHPHPPHPPHPPPPHPPPHPPHPPHPPPPHHPHPPHPPHHH

```

In []:

In []:

ISA-1 : PROTEIN FOLDING WITH TABULAR Q-LEARNING

Easy to read, step-by-step, heavily
commented code

Run this in Google Colab — one cell at a
time

```
In [6]: # -----  
# CELL 1 : Install libraries we need  
# -----  
# Run this first  
!pip install matplotlib numpy -q
```

```
In [7]: # -----  
# CELL 2 : Import everything  
# -----  
import numpy as np  
import matplotlib.pyplot as plt  
import matplotlib.patches as mpatches  
from matplotlib.colors import LinearSegmentedColormap  
import random  
import collections  
import os  
from IPython.display import clear_output, display  
import time  
  
print("✅ All libraries imported successfully!")
```

✅ All libraries imported successfully!

```
In [8]: # -----  
# CELL 3 : LOAD YOUR DATASET (the 2-lakh .txt file)  
# -----
```

```

def load_hp_dataset(filepath, max_len=30, min_len=10, sample_size=1000):
    """
    Load HP sequences from your 2-lakh .txt file.
    Each line in the file is one HP string like: HPHHPHHPHP...

    We pick short ones (10–30) for Q-Learning because
    Q-Table grows too large for longer chains.
    """
    sequences = []

    # — If you uploaded the file to Colab, change this path —
    # from google.colab import files
    # uploaded = files.upload() # <-- uncomment to upload
    # filepath = list(uploaded.keys())[0]

    if not os.path.exists(filepath):
        print(f"⚠ File not found: {filepath}")
        print(f"👉 Using built-in benchmark sequences instead.")
        # Famous Istrail Benchmark sequences (known optimal rewards)
        return [
            "HPHPPHHPHPPHHPHPPH",      # 20-mer, known optimum = 9
            "PHPHHPHHPHPPHHPH",        # 18-mer
            "HHHPPH",                   # 8-mer, known optimum = 2
            "HPPHPPHPPHPPHPPHPPH",     # 19-mer
            "HHPHPPHPPHPPHPPHPPHPPHH", # 24-mer
        ]

    with open(filepath, 'r') as f:
        for line in f:
            seq = line.strip().upper()
            # Only keep sequences in the desired length range
            if min_len <= len(seq) <= max_len and all(c in 'HP' for c in seq):
                sequences.append(seq)
            if len(sequences) >= sample_size:
                break

    print(f"✅ Loaded {len(sequences)} sequences (length {min_len}–{max_len})")
    return sequences

# — CHANGE THIS PATH to your uploaded file —————
DATASET_PATH = "hp_transformed_2laks.txt" # <-- update this!

all_sequences = load_hp_dataset(DATASET_PATH, max_len=25, min_len=8, sample_size=1000)
print(f"\n📄 First 5 sequences:")
for s in all_sequences[:5]:
    print(f"    {s} (length={len(s)})")

```

✓ Loaded 500 sequences (length 8–25)

First 5 sequences:
HHPHHPHPHPHPHPHPHPHPHPHPH (length=24)
PPPPHPHPHPHPHPHPHPHPHPH (length=22)
HPHHHPHPHPHPHPHPHPHPHPH (length=20)
HHPHPHPHPHPHPHPHPHPHPH (length=20)
PHPHPHPHPHPHPHPHPHPHPHH (length=21)

```
In [9]: # -----  
# CELL 4 : THE ENVIRONMENT – the grid where folding happens  
# -----  
  
class ProteinFoldingEnvironment:  
    """  
    This is the 'game board'.  
  
    Think of it as a 2D chess-board.  
    The agent places amino acid beads one at a time.  
    The chain must NEVER overlap itself (Self-Avoiding Walk).  
  
    Actions are RELATIVE (Left, Forward, Right) to prevent  
    the agent from going backward into itself.  
    """  
  
    # The 3 possible relative moves the agent can make  
    ACTIONS = ['Left', 'Forward', 'Right']  
  
    # How each move changes the (dx, dy) direction vector  
    # If currently moving East (+1,0):  
    #   Left → North (0,+1)  
    #   Forward → East (+1,0)  
    #   Right → South (0,-1)  
    TURN_LEFT = {'E':'N', 'N':'W', 'W':'S', 'S':'E'}  
    TURN_RIGHT = {'E':'S', 'S':'W', 'W':'N', 'N':'E'}  
    DIR_TO_DELTA = {'E':(1,0), 'W':(-1,0), 'N':(0,1), 'S':(0,-1)}  
  
    def __init__(self, hp_sequence):  
        self.sequence = hp_sequence # e.g. "HPPHH..."  
        self.length = len(hp_sequence)  
        self.reset()  
  
    def reset(self):  
        """  
        Put the first bead at (0,0), second bead at (1,0).  
        Agent starts facing East.  
        Returns the first state.  
        """  
        self.positions = [(0, 0), (1, 0)] # list of (x,y) for each bead placed  
        self.current_dir = 'E' # facing East  
        self.step_index = 2 # next bead to place is index 2  
        self.done = False  
        return self._get_state()
```

```

def _get_state(self):
    """
    Convert current chain into a simple hashable state for the Q-Table.
    We encode the last few relative turns taken – compact and efficient.
    """
    # Use the last 4 moves as state (good balance of memory vs. info)
    return tuple(self.positions[-4:]) if len(self.positions) >= 4 else tuple()

def step(self, action_index):
    """
    Take one action: place the next bead in a direction.

    Returns:
        next_state – new state after the move
        reward      – points earned (H-H contacts formed)
        done        – True if sequence is complete or chain collided
        info        – extra info for debugging
    """
    if self.done:
        return self._get_state(), 0, True, {}

    action = self.ACTIONS[action_index]

    # — Step 1: Update direction based on action —
    if action == 'Left':    self.current_dir = self.TURN_LEFT[self.current_dir]
    elif action == 'Right': self.current_dir = self.TURN_RIGHT[self.current_dir]
    # 'Forward' keeps the same direction

    # — Step 2: Calculate new bead position —
    dx, dy = self.DIR_TO_DELTA[self.current_dir]
    last_x, last_y = self.positions[-1]
    new_pos = (last_x + dx, last_y + dy)

    # — Step 3: Check collision (Self-Avoiding Walk) —
    if new_pos in self.positions:
        # Chain collided with itself – episode ends with penalty
        self.done = True
        return self._get_state(), -1, True, {'reason': 'collision'}

    # — Step 4: Place the bead —
    self.positions.append(new_pos)
    bead_type = self.sequence[self.step_index] # 'H' or 'P'
    self.step_index += 1

    # — Step 5: Calculate reward (count new H-H contacts) —
    reward = self._count_new_hh_contacts(new_pos, self.step_index - 1)

    # — Step 6: Check if we finished placing all beads —
    if self.step_index >= self.length:
        self.done = True

    return self._get_state(), reward, self.done, {}

```

```

def _count_new_hh_contacts(self, new_pos, new_index):
    """
    Count how many NEW H-H contacts are formed by placing this bead.

    A contact is valid if:
    1. The neighbor is an H bead
    2. The new bead is also H
    3. They are NOT adjacent in the sequence ( $|i-j| > 1$ )
        This ensures it's a STRUCTURAL contact, not just a bond
    """
    if self.sequence[new_index] != 'H':
        return 0 # Only H beads can form hydrophobic contacts

    reward = 0
    x, y = new_pos
    neighbors = [(x+1,y), (x-1,y), (x,y+1), (x,y-1)]

    for nx, ny in neighbors:
        if (nx, ny) not in self.positions[:-1]:
            continue # No bead at this neighbor position

        # Find which bead index occupies this neighbor position
        neighbor_index = self.positions.index((nx, ny))

        # Check: is the neighbor an H bead?
        if self.sequence[neighbor_index] != 'H':
            continue

        # Check: are they non-adjacent in the sequence?
        if abs(new_index - neighbor_index) > 1:
            reward += 1 # Valid H-H contact!

    return reward

def get_total_contacts(self):
    """Count total H-H contacts in the current configuration."""
    total = 0
    occupied = {pos: i for i, pos in enumerate(self.positions)}

    for i, pos in enumerate(self.positions):
        if self.sequence[i] != 'H':
            continue
        x, y = pos
        for nx, ny in [(x+1,y), (x-1,y), (x,y+1), (x,y-1)]:
            if (nx,ny) in occupied:
                j = occupied[(nx,ny)]
                if j > i and abs(i-j) > 1 and self.sequence[j] == 'H':
                    total += 1
    return total

print("✅ Environment class defined!")

```

✅ Environment class defined!

In [10]:

```
# -----  
# CELL 5 : THE Q-LEARNING AGENT  
# -----  
  
class QLearningAgent:  
    """  
    The Q-Learning Agent learns by building a Q-Table.  
  
    Q-Table: A dictionary where  
    Key   = (state, action)  
    Value = expected future reward from taking that action in that state  
  
    The agent:  
    - EXPLORES randomly early on (high epsilon)  
    - EXPLOITS its learned knowledge later (low epsilon)  
    """  
  
    def __init__(self,  
                  n_actions      = 3,      # Left, Forward, Right  
                  learning_rate  = 0.1,    # How fast to update Q-values  
                  discount       = 0.95,   # How much to value future rewards  
                  epsilon        = 1.0,    # Starting exploration rate (100% random)  
                  epsilon_decay  = 0.995,  # How fast exploration decreases  
                  epsilon_min    = 0.05):  # Minimum exploration (always explore)  
  
        self.n_actions      = n_actions  
        self.lr             = learning_rate  
        self.gamma          = discount  
        self.epsilon        = epsilon  
        self.epsilon_decay  = epsilon_decay  
        self.epsilon_min    = epsilon_min  
  
        # The Q-Table: state → [Q-value for action 0, Q-value for action 1, Q-value for action 2]  
        self.q_table = collections.defaultdict(lambda: np.zeros(n_actions))  
  
    def choose_action(self, state):  
        """  
        Epsilon-Greedy Strategy:  
        - With probability epsilon → pick a random action (EXPLORE)  
        - Otherwise → pick the best known action (EXPLOIT)  
        """  
        if random.random() < self.epsilon:  
            return random.randint(0, self.n_actions - 1) # Random action  
        else:  
            return np.argmax(self.q_table[state]) # Best known action  
  
    def learn(self, state, action, reward, next_state, done):  
        """  
        Update the Q-Table using the Bellman Equation:  
  
         $Q(S, A) \leftarrow Q(S, A) + \alpha \times [R + \gamma \times \max_{A'} Q(S', A') - Q(S, A)]$   
        ↑ current      ↑lr   ↑reward ↑future value ↑current estimate  
        """
```

```

        current_q = self.q_table[state][action]

        if done:
            target_q = reward # No future rewards if episode is over
        else:
            best_future_q = np.max(self.q_table[next_state])
            target_q = reward + self.gamma * best_future_q

        # Update rule (move Q-value toward the target)
        self.q_table[state][action] += self.lr * (target_q - current_q)

    def decay_epsilon(self):
        """Reduce exploration rate after each episode."""
        self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay)

    @property
    def q_table_size(self):
        return len(self.q_table)

print("✅ Q-Learning Agent class defined!")

```

✅ Q-Learning Agent class defined!

```

In [11]: # -----
# CELL 6 : TRAIN THE AGENT
# -----

def train_q_agent(hp_sequence, n_episodes=3000, verbose=True):
    """
    Train a Q-Learning agent on ONE protein sequence.

    For each episode:
        1. Reset the environment (start fresh)
        2. Agent takes actions until done
        3. Agent learns from each (state, action, reward, next_state) tuple
        4. Repeat
    """
    env = ProteinFoldingEnvironment(hp_sequence)
    agent = QLearningAgent()

    # Tracking metrics for graphs
    episode_rewards = []
    episode_contacts = []
    best_reward = -999
    best_positions = None
    rolling_avg = []

    print(f"\n{'='*55}")
    print(f" Training on sequence: {hp_sequence}")
    print(f" Length: {len(hp_sequence)} | Episodes: {n_episodes}")
    print(f"{'='*55}")

    for episode in range(n_episodes):

```

```

# — Reset environment for new episode —
state = env.reset()
total_reward = 0
done = False

# — Run one full episode —
while not done:
    action = agent.choose_action(state)
    next_state, reward, done, _ = env.step(action)
    agent.learn(state, action, reward, next_state, done)
    state = next_state
    total_reward += reward

# — Track the best fold found —
contacts = env.get_total_contacts()
if contacts > best_reward:
    best_reward = contacts
    best_positions = list(env.positions)

episode_rewards.append(total_reward)
episode_contacts.append(contacts)

# — Reduce exploration rate —
agent.decay_epsilon()

# — Rolling average for smooth graph —
window = min(100, len(episode_contacts))
rolling_avg.append(np.mean(episode_contacts[-window:]))

# — Print progress every 500 episodes —
if verbose and (episode + 1) % 500 == 0:
    print(f" Episode {episode+1:4d} | "
          f"Avg Contacts (last 100): {rolling_avg[-1]:.2f} | "
          f"Best: {best_reward} | "
          f"Epsilon: {agent.epsilon:.3f} | "
          f"Q-Table size: {agent.q_table_size}")

print(f"\n✅ Training complete! Best H-H contacts found: {best_reward}")
return agent, best_positions, episode_contacts, rolling_avg, hp_sequence

# — Pick your sequence —
TARGET_SEQUENCE = "HPHPPHHPHPPHHPHPPH" # Famous 20-mer benchmark (known to
# You can also pick from your dataset:
# TARGET_SEQUENCE = all_sequences[0]

agent, best_positions, contacts_per_ep, rolling_avg, sequence = train_q_agent(
    TARGET_SEQUENCE,
    n_episodes = 5000
)

```

```
=====
Training on sequence: HPHPPHHPHPPHHPHPPH
Length: 20 | Episodes: 5000
=====
```

```
Episode 500 | Avg Contacts (last 100): 4.02 | Best: 6 | Epsilon: 0.082 | Q-T
able size: 940
Episode 1000 | Avg Contacts (last 100): 5.03 | Best: 6 | Epsilon: 0.050 | Q-T
able size: 973
Episode 1500 | Avg Contacts (last 100): 4.81 | Best: 6 | Epsilon: 0.050 | Q-T
able size: 988
Episode 2000 | Avg Contacts (last 100): 4.85 | Best: 6 | Epsilon: 0.050 | Q-T
able size: 1001
Episode 2500 | Avg Contacts (last 100): 4.65 | Best: 6 | Epsilon: 0.050 | Q-T
able size: 1018
Episode 3000 | Avg Contacts (last 100): 4.95 | Best: 6 | Epsilon: 0.050 | Q-T
able size: 1027
Episode 3500 | Avg Contacts (last 100): 4.68 | Best: 6 | Epsilon: 0.050 | Q-T
able size: 1051
Episode 4000 | Avg Contacts (last 100): 4.76 | Best: 6 | Epsilon: 0.050 | Q-T
able size: 1057
Episode 4500 | Avg Contacts (last 100): 5.16 | Best: 6 | Epsilon: 0.050 | Q-T
able size: 1073
Episode 5000 | Avg Contacts (last 100): 5.00 | Best: 6 | Epsilon: 0.050 | Q-T
able size: 1079
```

✓ Training complete! Best H-H contacts found: 6

```
In [12]: # -----
# CELL 7 : TRAINING GRAPHS
# -----

def plot_training_graphs(contacts_per_ep, rolling_avg, sequence):
    """Draw 3 training analysis graphs side by side."""

    fig, axes = plt.subplots(1, 3, figsize=(18, 5))
    fig.suptitle(f"ISA-1 Q-Learning Training Analysis\nSequence: {sequence}",
                 fontsize=14, fontweight='bold', y=1.02)

    episodes = range(len(contacts_per_ep))
    color_raw = '#a8d8ea'
    color_avg = '#e94560'
    color_dist = '#0f3460'

    # — Graph 1: Contacts per Episode —————
    ax1 = axes[0]
    ax1.plot(episodes, contacts_per_ep, color=color_raw, alpha=0.4,
              linewidth=0.8, label='H-H Contacts per Episode')
    ax1.plot(episodes, rolling_avg, color=color_avg,
              linewidth=2.5, label='Rolling Average (100 ep)')
    ax1.axhline(y=max(contacts_per_ep), color='green', linestyle='--',
                 linewidth=1.5, label=f'Best Found: {max(contacts_per_ep)}')
    ax1.set_xlabel('Episode', fontsize=11)
    ax1.set_ylabel('H-H Contacts (Reward)', fontsize=11)
    ax1.set_title('Learning Progress Over Time', fontsize=12, fontweight='bold')
```

```

ax1.legend(fontsize=9)
ax1.grid(True, alpha=0.3)
ax1.set_facecolor('#f8f9fa')

# — Graph 2: Distribution of rewards —————
ax2 = axes[1]
ax2.hist(contacts_per_ep, bins=max(contacts_per_ep)+2,
         color=color_dist, edgecolor='white', alpha=0.85,
         range=(-1, max(contacts_per_ep)+1))
ax2.axvline(x=np.mean(contacts_per_ep), color=color_avg, linewidth=2,
           linestyle='--', label=f'Mean: {np.mean(contacts_per_ep):.2f}')
ax2.set_xlabel('H-H Contacts per Episode', fontsize=11)
ax2.set_ylabel('Frequency (How Often)', fontsize=11)
ax2.set_title('Distribution of Episode Rewards', fontsize=12, fontweight='bold')
ax2.legend(fontsize=10)
ax2.grid(True, alpha=0.3)
ax2.set_facecolor('#f8f9fa')

# — Graph 3: Improvement over training segments —————
ax3 = axes[2]
n = len(contacts_per_ep)
seg_size = n // 10
segment_avgs = [np.mean(contacts_per_ep[i*seg_size:(i+1)*seg_size])
                for i in range(10)]
segment_labels = [f"{i*10}—{(i+1)*10}%" for i in range(10)]
bars = ax3.bar(segment_labels, segment_avgs,
               color=plt.cm.RdYlGn(np.linspace(0.2, 0.9, 10)),
               edgecolor='white', linewidth=0.8)
ax3.set_xlabel('Training Progress (% Complete)', fontsize=11)
ax3.set_ylabel('Average H-H Contacts', fontsize=11)
ax3.set_title('Average Reward by Training Stage', fontsize=12, fontweight='bold')
ax3.tick_params(axis='x', rotation=45)
ax3.grid(True, alpha=0.3, axis='y')
ax3.set_facecolor('#f8f9fa')
for bar, val in zip(bars, segment_avgs):
    ax3.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
             f'{val:.1f}', ha='center', va='bottom', fontsize=8)

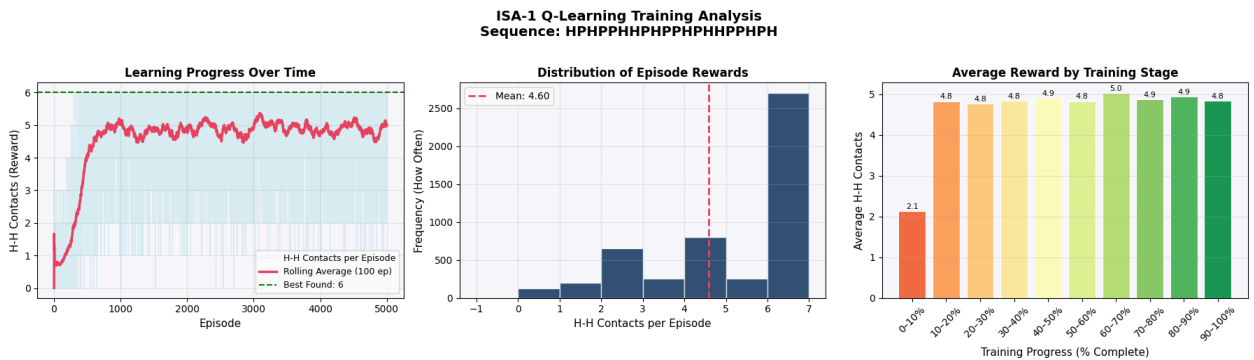
plt.tight_layout()
plt.savefig('isa1_training_graphs.png', dpi=150, bbox_inches='tight',
          facecolor='white')
plt.show()
print("📊 Training graphs saved as 'isa1_training_graphs.png'")

```

```

plot_training_graphs(contacts_per_ep, rolling_avg, sequence)

```



Training graphs saved as 'isa1_training_graphs.png'

```
In [13]: # -----
# CELL 8 : VISUALIZE THE BEST FOLD (2D + Animated)
# -----

def visualize_best_fold_2d(positions, sequence, title="Best Protein Fold Found")
    """
    Draw the best fold the agent discovered on a 2D grid.

    Color coding:
        ● Red   = H (Hydrophobic) bead
        ● Blue  = P (Polar) bead
        Green lines = H-H contacts (the reward-giving connections)
    """
    fig, ax = plt.subplots(1, 1, figsize=(10, 10))

    xs = [p[0] for p in positions]
    ys = [p[1] for p in positions]

    # — Draw the backbone (chain bonds) —
    ax.plot(xs, ys, color='#555555', linewidth=3, zorder=1,
            solid_capstyle='round', label='Backbone chain')

    # — Draw H-H contact lines —
    occupied = {pos: i for i, pos in enumerate(positions)}
    contact_count = 0
    for i, pos in enumerate(positions):
        if sequence[i] != 'H':
            continue
        x, y = pos
        for nx, ny in [(x+1,y), (x-1,y), (x,y+1), (x,y-1)]:
            if (nx,ny) in occupied:
                j = occupied[(nx,ny)]
                if j > i and abs(i-j) > 1 and sequence[j] == 'H':
                    ax.plot([x, nx], [y, ny],
                            color='#2ecc71', linewidth=4,
                            linestyle='--', alpha=0.8, zorder=2)
                    contact_count += 1

    # — Draw each bead —
    for i, (x, y) in enumerate(positions):
        bead_type = sequence[i]
```

```

color      = '#e74c3c' if bead_type == 'H' else '#3498db'
size       = 500 if bead_type == 'H' else 400

# Outer ring for visibility
ax.scatter(x, y, s=size + 200, color='white', zorder=3)
ax.scatter(x, y, s=size, color=color, zorder=4,
           edgecolors='#2c3e50', linewidths=2)

# Label the bead
ax.annotate(f'{bead_type}\n{i+1}',
           xy=(x, y), ha='center', va='center',
           fontsize=8, fontweight='bold', color='white', zorder=5)

# — Mark start and end —————
ax.scatter(*positions[0], s=700, marker='*', color='#f39c12', zorder=6,
           edgecolors='black', linewidths=1.5, label='Start (bead 1)')
ax.scatter(*positions[-1], s=700, marker='D', color='#9b59b6', zorder=6,
           edgecolors='black', linewidths=1.5, label='End (last bead)')

# — Styling —————
margin = 2
ax.set_xlim(min(xs)-margin, max(xs)+margin)
ax.set_ylim(min(ys)-margin, max(ys)+margin)
ax.set_aspect('equal')
ax.grid(True, alpha=0.2, linestyle=':')
ax.set_facecolor('#f0f4f8')
ax.set_title(f'{title}\nSequence: {sequence}\n'
             f'H-H Contacts (Reward) = {contact_count}',
             fontsize=13, fontweight='bold', pad=15)

# — Legend —————
legend_elements = [
    mpatches.Patch(color='#e74c3c', label='H = Hydrophobic (water-fearing)'),
    mpatches.Patch(color='#3498db', label='P = Polar (water-loving)'),
    mpatches.Patch(color='#2ecc71', label=f'H-H Contact (+1 reward each) ×'),
    mpatches.Patch(color='#f39c12', label='★ Start (N-terminus)'),
    mpatches.Patch(color='#9b59b6', label='◆ End (C-terminus)'),
]
ax.legend(handles=legend_elements, loc='upper right',
          fontsize=10, framealpha=0.95)

plt.tight_layout()
plt.savefig('isa1_best_fold_2d.png', dpi=150, bbox_inches='tight',
           facecolor='white')
plt.show()
print(f"📁 2D fold saved as 'isa1_best_fold_2d.png'")
print(f"🏆 Total H-H Contacts: {contact_count}")

```

```
visualize_best_fold_2d(best_positions, sequence)
```

```

# —————
# CELL 9 : ANIMATED FOLDING – Watch the protein fold step by step

```

```

# -----
def animate_folding_steps(positions, sequence):
    """
    Show how the protein was built step by step.
    Each frame adds one more bead to the chain.
    Great for understanding the sequential decision-making process.
    """
    n = len(positions)
    cols = min(5, n)
    rows = (n + cols - 1) // cols

    fig, axes = plt.subplots(rows, cols,
                             figsize=(cols * 3.5, rows * 3.5))
    fig.suptitle(f'Step-by-Step Protein Folding Process\nSequence: {sequence}',
                 fontsize=14, fontweight='bold')

    # Flatten axes array for easy iteration
    axes_flat = axes.flatten() if hasattr(axes, 'flatten') else [axes]

    all_xs = [p[0] for p in positions]
    all_ys = [p[1] for p in positions]
    x_min, x_max = min(all_xs)-1, max(all_xs)+1
    y_min, y_max = min(all_ys)-1, max(all_ys)+1

    for step in range(n):
        ax = axes_flat[step]

        # Draw beads placed so far
        current_positions = positions[:step+1]
        xs = [p[0] for p in current_positions]
        ys = [p[1] for p in current_positions]

        # Draw chain
        if len(current_positions) > 1:
            ax.plot(xs, ys, color='#555555', linewidth=2, zorder=1)

        # Draw beads
        for i, (x, y) in enumerate(current_positions):
            color = '#e74c3c' if sequence[i] == 'H' else '#3498db'
            ax.scatter(x, y, s=300, color=color, zorder=3,
                      edgecolors='#2c3e50', linewidths=1.5)
            ax.annotate(sequence[i], xy=(x,y), ha='center', va='center',
                       fontsize=9, fontweight='bold', color='white', zorder=4)

        # Highlight current (newest) bead
        ax.scatter(*current_positions[-1], s=400,
                  color='#f39c12', zorder=5,
                  edgecolors='#2c3e50', linewidths=2)

        ax.set_xlim(x_min, x_max)
        ax.set_ylim(y_min, y_max)
        ax.set_aspect('equal')

```

```

ax.grid(True, alpha=0.2, linestyle=':')
ax.set_facecolor('#f8f9fa')
ax.set_title(f'Step {step+1}: Place "{sequence[step]}"',
             fontsize=9, fontweight='bold')
ax.tick_params(labelsize=6)

# Hide unused subplots
for i in range(n, len(axes_flat)):
    axes_flat[i].set_visible(False)

plt.tight_layout()
plt.savefig('isa1_folding_steps.png', dpi=130, bbox_inches='tight',
           facecolor='white')
plt.show()
print("🎬 Step-by-step folding image saved as 'isa1_folding_steps.png'")

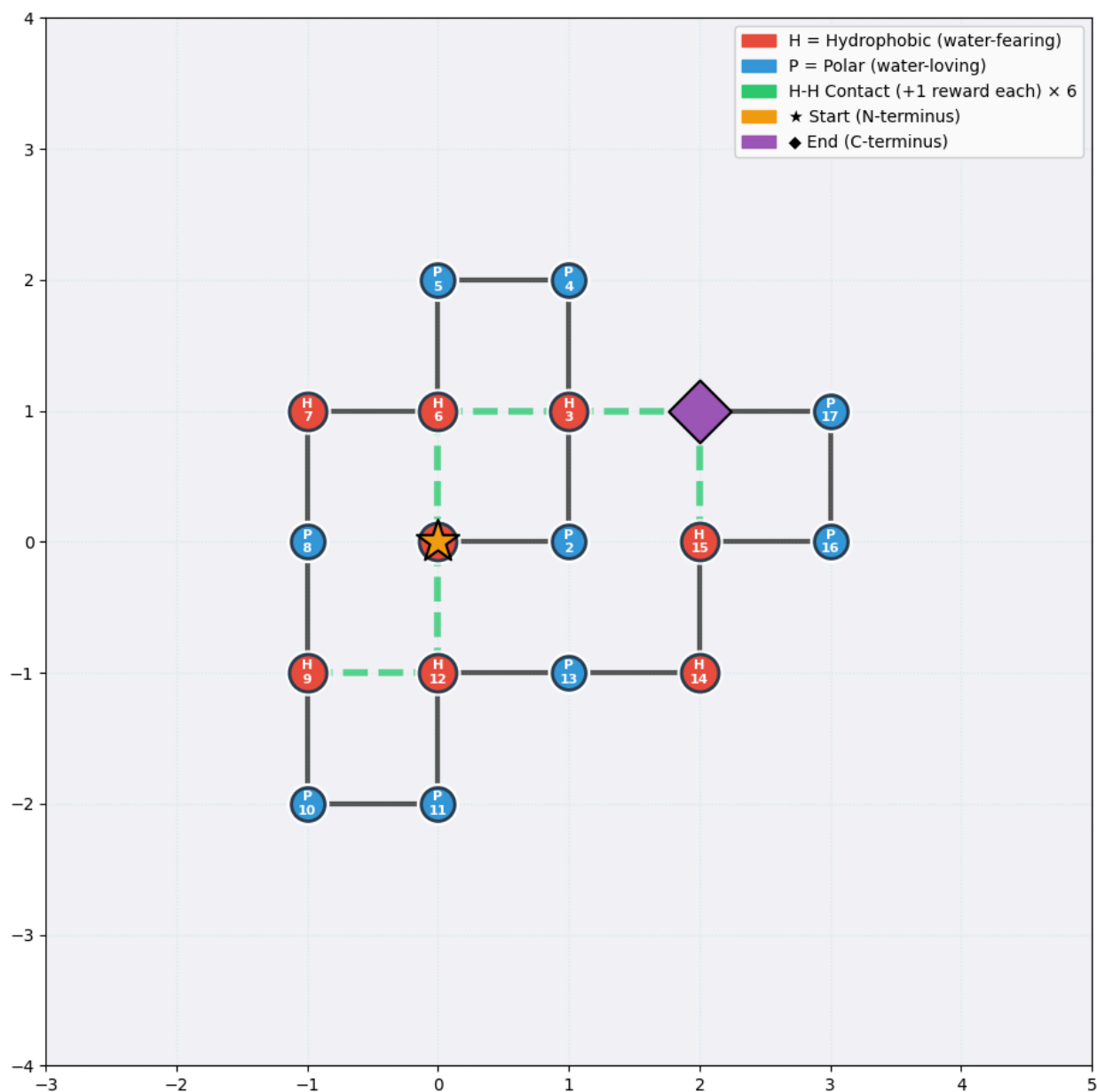
```

```

animate_folding_steps(best_positions, sequence)

```

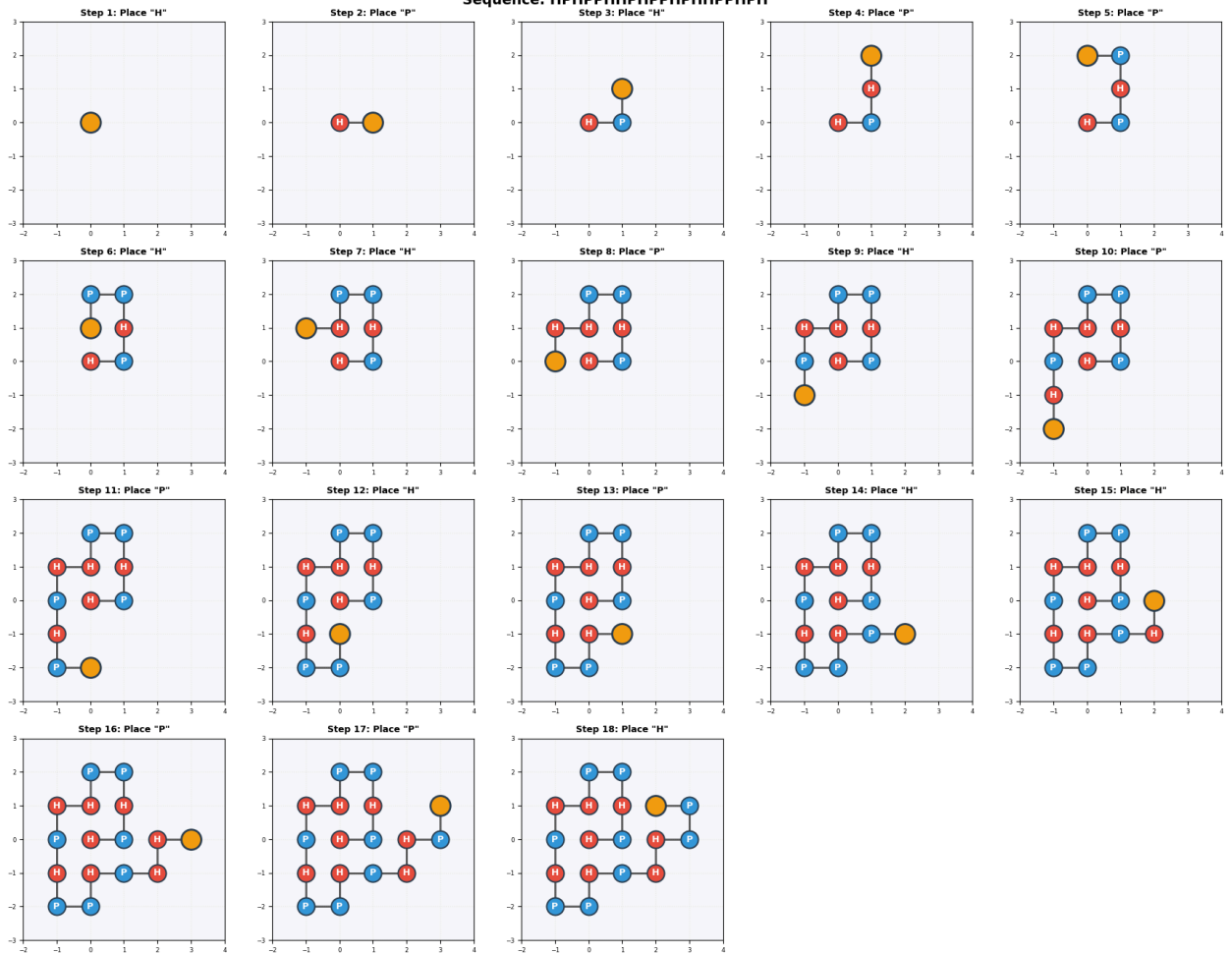
Best Protein Fold Found
Sequence: HPHPPHHPHPPHPPHPPHPPH
H-H Contacts (Reward) = 6



📷 2D fold saved as 'isa1_best_fold_2d.png'

🏆 Total H-H Contacts: 6

Step-by-Step Protein Folding Process Sequence: HPHPPHHPHPPHPPHPPHPPH



Step-by-step folding image saved as 'isa1_folding_steps.png'

```
In [14]: # _____
# CELL 10 : TEST ON MULTIPLE SEQUENCES (batch evaluation)
# _____

def test_multiple_sequences(sequences, n_episodes=2000):
    """
    Train the agent on several sequences and compare results.
    """
    results = []
    for seq in sequences[:8]: # Test on first 8
        if len(seq) > 25:
            continue # Skip long ones for speed
        print(f"\n🔄 Training on: {seq} (len={len(seq)})")
        agent, best_pos, ep_contacts, _, _ = train_q_agent(
            seq, n_episodes=n_episodes, verbose=False)
        best = max(ep_contacts)
        avg = np.mean(ep_contacts[-200:])
        results.append({
            'sequence' : seq[:15] + '...' if len(seq)>15 else seq,
            'length' : len(seq),
            'best' : best,
            'avg_last200' : avg
```

```

    })
    print(f"    ✅ Best contacts: {best} | Avg (last 200 ep): {avg:.2f}")

# Summary bar chart
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('ISA-1: Performance Across Multiple Sequences',
             fontsize=13, fontweight='bold')

seqs    = [r['sequence'] for r in results]
bests   = [r['best'] for r in results]
avgs    = [r['avg_last200'] for r in results]

x = np.arange(len(seqs))
ax1.bar(x, bests, color='#e74c3c', alpha=0.8, label='Best Contacts Found')
ax1.bar(x, avgs, color='#3498db', alpha=0.8, label='Avg Last 200 Episodes',
        width=0.5)
ax1.set_xticks(x)
ax1.set_xticklabels(seqs, rotation=45, ha='right', fontsize=8)
ax1.set_ylabel('H-H Contacts')
ax1.set_title('Best vs Average Performance')
ax1.legend()
ax1.grid(True, alpha=0.3, axis='y')

lengths = [r['length'] for r in results]
ax2.scatter(lengths, bests, color='#e74c3c', s=120,
            zorder=3, label='Best Contacts')
ax2.scatter(lengths, avgs, color='#3498db', s=80,
            marker='D', zorder=3, label='Avg Last 200')
ax2.set_xlabel('Sequence Length')
ax2.set_ylabel('H-H Contacts')
ax2.set_title('Performance vs Sequence Length')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('isa1_multi_sequence_results.png', dpi=130, bbox_inches='tight',
          facecolor='white')
plt.show()
print("📊 Multi-sequence results saved!")
return results

results = test_multiple_sequences(all_sequences, n_episodes=1500)

print("\n" + "="*55)
print("  ISA-1 COMPLETE! Summary of what was created:")
print("="*55)
print("  📊 isa1_training_graphs.png      – Learning curves")
print("  🌞 isa1_best_fold_2d.png         – Best fold visualization")
print("  🎬 isa1_folding_steps.png        – Step-by-step folding")
print("  📊 isa1_multi_sequence_results.png – Batch evaluation")
print("="*55)

```

🔄 Training on: HHPHHPHPPHPHHPHPPPPHHPH (len=24)

=====
Training on sequence: HHPHHPHPPHPHHPHPPPPHHPH
Length: 24 | Episodes: 1500
=====

✓ Training complete! Best H-H contacts found: 8
✓ Best contacts: 8 | Avg (last 200 ep): 5.78

🔄 Training on: PPPPPHPHHPHPHPPHHPHHP (len=22)

=====
Training on sequence: PPPPPHPHHPHPHPPHHPHHP
Length: 22 | Episodes: 1500
=====

✓ Training complete! Best H-H contacts found: 4
✓ Best contacts: 4 | Avg (last 200 ep): 1.73

🔄 Training on: HPHHHPHPPHPPHHPHHPHPP (len=20)

=====
Training on sequence: HPHHHPHPPHPPHHPHHPHPP
Length: 20 | Episodes: 1500
=====

✓ Training complete! Best H-H contacts found: 5
✓ Best contacts: 5 | Avg (last 200 ep): 3.21

🔄 Training on: HHPHHPHPPHPPHPPPPHHH (len=20)

=====
Training on sequence: HHPHHPHPPHPPHPPPPHHH
Length: 20 | Episodes: 1500
=====

✓ Training complete! Best H-H contacts found: 7
✓ Best contacts: 7 | Avg (last 200 ep): 4.52

🔄 Training on: PHPHPPPPHHPHPPHHPHHPH (len=21)

=====
Training on sequence: PHPHPPPPHHPHPPHHPHHPH
Length: 21 | Episodes: 1500
=====

✓ Training complete! Best H-H contacts found: 4
✓ Best contacts: 4 | Avg (last 200 ep): 1.50

🔄 Training on: PHPHPPPPHHPHPPHHPHHPH (len=21)

=====
Training on sequence: PHPHPPPPHHPHPPHHPHHPH

=====

=====

=====

=====

=====

=====

=====

=====

ISA-2 : PROTEIN FOLDING WITH DOUBLE DEEP Q-NETWORK (DDQN)

Scales to 200,000 sequences from your UniProt dataset

Run in Google Colab — GPU recommended (Runtime → Change runtime → T4 GPU)

```
In [17]: # -----  
# CELL 1 : Install and import libraries  
# -----  
!pip install torch numpy matplotlib plotly -q  
  
import numpy as np  
import matplotlib.pyplot as plt  
import matplotlib.patches as mpatches  
from mpl_toolkits.mplot3d import Axes3D  
from mpl_toolkits.mplot3d.art3d import Line3DCollection  
import plotly.graph_objects as go  
import plotly.express as px  
from plotly.subplots import make_subplots  
import random  
import collections  
import os  
import math  
  
import torch  
import torch.nn as nn  
import torch.optim as optim  
  
# Use GPU if available (makes training much faster)  
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
print(f"✅ Using device: {DEVICE}")  
print(f"PyTorch version: {torch.__version__}")
```

✓ Using device: cpu
PyTorch version: 2.10.0+cpu

```
In [18]: # -----  
# CELL 2 : LOAD YOUR 2-LAKH DATASET  
# -----  
  
def load_dataset(filepath, min_len=10, max_len=50, sample_size=200000):  
    """  
    Load all HP sequences from your 2-lakh .txt file.  
    For ISA-2 we can handle longer sequences (up to 50).  
    """  
  
    # — TO UPLOAD YOUR FILE IN COLAB: -----  
    # from google.colab import files  
    # uploaded = files.upload()  
    # filepath = list(uploaded.keys())[0]  
  
    if not os.path.exists(filepath):  
        print(f"⚠ File '{filepath}' not found.")  
        print("🚀 Generating a sample dataset of 5000 synthetic HP sequences..")  
        return _generate_sample_dataset(5000, min_len, max_len)  
  
    sequences = []  
    with open(filepath, 'r') as f:  
        for line in f:  
            seq = line.strip().upper()  
            if min_len <= len(seq) <= max_len and all(c in 'HP' for c in seq):  
                sequences.append(seq)  
            if len(sequences) >= sample_size:  
                break  
  
    random.shuffle(sequences)  
    print(f"✓ Loaded {len(sequences)} sequences from '{filepath}'")  
    print(f"Length range: {min(len(s) for s in sequences)} – {max(len(s) for s in sequences)}")  
    return sequences  
  
def _generate_sample_dataset(n, min_len, max_len):  
    """Generate synthetic HP sequences if file not available."""  
    seqs = []  
    for _ in range(n):  
        length = random.randint(min_len, max_len)  
        # Bias toward 40% H to match real protein statistics  
        seq = ''.join(random.choices('HP', weights=[40, 60], k=length))  
        seqs.append(seq)  
    return seqs  
  
# — CHANGE THIS to your file path -----  
DATASET_PATH = "hp_sequences.txt" # <-- update this!  
  
all_sequences = load_dataset(DATASET_PATH, min_len=10, max_len=50, sample_size=
```

```
# — Split into train / validation —————
split      = int(0.9 * len(all_sequences))
train_seqs = all_sequences[:split]
val_seqs   = all_sequences[split:]

print(f"\n📁 Training sequences: {len(train_seqs)}")
print(f"📁 Validation sequences: {len(val_seqs)}")
```

⚠ File 'hp_sequences.txt' not found.

🔥 Generating a sample dataset of 5000 synthetic HP sequences...

📁 Training sequences: 4500

📁 Validation sequences: 500

```
In [19]: # —————
# CELL 3 : ENVIRONMENT (same logic as ISA-1, but extended for 3D)
# —————

class ProteinFoldingEnv3D:
    """
    A 3D protein folding environment.
    The protein chain is built bead by bead on a 3D lattice (x, y, z grid).

    In 3D we have more actions: Forward, Left, Right, Up, Down
    (relative to current movement direction). This gives 5 choices.
    Still using relative actions to prevent 180° backtracking.
    """

    N_ACTIONS = 5 # Forward, Left, Right, Up, Down (relative)

    def __init__(self, hp_sequence):
        self.sequence = hp_sequence
        self.length   = len(hp_sequence)
        self.obs_size  = self._compute_obs_size()
        self.reset()

    def _compute_obs_size(self):
        """
        The observation (what the agent sees) is a local window
        of size 5x5x5 = 125 cells around the current head position.
        Each cell has 3 possible values: empty=0, H=1, P=2.
        Plus: the types of the next 3 beads to place (one-hot).
        """
        return 125 + 3 # 5x5x5 window + 3 upcoming bead types

    def reset(self):
        self.positions = [(0, 0, 0), (1, 0, 0)]
        self.direction = (1, 0, 0) # currently moving in +X direction
        self.step_index = 2
        self.done       = False
        self.pos_set     = set(self.positions)
        return self._get_observation()

# — 3D direction rotation helpers —————
```

```

def _turn_left(self, d):
    x, y, z = d
    return (-y, x, z) # rotate around Z-axis

def _turn_right(self, d):
    x, y, z = d
    return (y, -x, z) # rotate around Z-axis

def _turn_up(self, d):
    x, y, z = d
    return (-z, y, x) # rotate around Y-axis

def _turn_down(self, d):
    x, y, z = d
    return (z, y, -x) # rotate around Y-axis

def step(self, action):
    """
    Place the next bead in the chosen direction.
    Actions: 0=Forward, 1=Left, 2=Right, 3=Up, 4=Down
    """
    if self.done:
        return self._get_observation(), 0, True, {}

    # — Compute new direction —————
    d = self.direction
    if action == 1: d = self._turn_left(d)
    elif action == 2: d = self._turn_right(d)
    elif action == 3: d = self._turn_up(d)
    elif action == 4: d = self._turn_down(d)
    # action == 0: keep direction (Forward)
    self.direction = d

    # — Compute new bead position —————
    hx, hy, hz = self.positions[-1]
    new_pos = (hx + d[0], hy + d[1], hz + d[2])

    # — Collision check —————
    if new_pos in self.pos_set:
        self.done = True
        return self._get_observation(), -1, True, {'reason': 'collision'}

    self.positions.append(new_pos)
    self.pos_set.add(new_pos)
    current_bead_idx = self.step_index
    bead_type = self.sequence[current_bead_idx]
    self.step_index += 1

    # — Reward (H-H contacts) —————
    reward = self._new_hh_contacts(new_pos, current_bead_idx)

    if self.step_index >= self.length:
        self.done = True

```

```

        return self._get_observation(), reward, self.done, {}

def _new_hh_contacts(self, pos, idx):
    """Count new H-H contacts formed by placing bead at pos with sequence
    if self.sequence[idx] != 'H':
        return 0
    reward = 0
    x, y, z = pos
    # Check all 6 adjacent cells in 3D
    for dx, dy, dz in [(1,0,0),(-1,0,0),(0,1,0),(0,-1,0),(0,0,1),(0,0,-1)]:
        neighbor = (x+dx, y+dy, z+dz)
        if neighbor not in self.pos_set:
            continue
        j = self.positions.index(neighbor)
        if self.sequence[j] == 'H' and abs(idx - j) > 1:
            reward += 1
    return reward

def get_total_contacts(self):
    """Count total H-H contacts in current configuration."""
    total = 0
    pos_dict = {pos: i for i, pos in enumerate(self.positions)}
    for i, pos in enumerate(self.positions):
        if self.sequence[i] != 'H': continue
        x, y, z = pos
        for dx, dy, dz in [(1,0,0),(-1,0,0),(0,1,0),(0,-1,0),(0,0,1),(0,0,-1)]:
            nb = (x+dx, y+dy, z+dz)
            if nb in pos_dict:
                j = pos_dict[nb]
                if j > i and abs(i-j) > 1 and self.sequence[j] == 'H':
                    total += 1
    return total

def _get_observation(self):
    """
    Local 5x5x5 observation window centered on the current head.
    This is what the neural network 'sees'.
    """
    obs = np.zeros(125 + 3, dtype=np.float32)
    hx, hy, hz = self.positions[-1]
    pos_dict = {pos: i for i, pos in enumerate(self.positions)}

    # Fill the 5x5x5 window
    idx = 0
    for dz in range(-2, 3):
        for dy in range(-2, 3):
            for dx in range(-2, 3):
                cell = (hx+dx, hy+dy, hz+dz)
                if cell in pos_dict:
                    bi = pos_dict[cell]
                    obs[idx] = 1.0 if self.sequence[bi] == 'H' else 0.5
                idx += 1

```

```

    # Next 3 bead types (upcoming decisions)
    for k in range(3):
        si = self.step_index + k
        if si < self.length:
            obs[125 + k] = 1.0 if self.sequence[si] == 'H' else 0.5

    return obs

print("✅ 3D Environment class defined!")

```

✅ 3D Environment class defined!

```

In [20]: # -----
# CELL 4 : THE NEURAL NETWORK (the 'brain' of the DQN agent)
# -----

class DQNNetwork(nn.Module):
    """
    A simple 3-layer neural network.

    Input → Hidden Layer 1 → Hidden Layer 2 → Hidden Layer 3 → Output

    Input: the observation (what the agent sees around it)
    Output: one Q-value for each possible action

    The agent picks the action with the HIGHEST Q-value.
    """

    def __init__(self, obs_size, n_actions, hidden=256):
        super(DQNNetwork, self).__init__()

        self.network = nn.Sequential(
            # Layer 1: Input → 256 neurons
            nn.Linear(obs_size, hidden),
            nn.ReLU(), # Activation: "fire only if positive"
            nn.LayerNorm(hidden), # Normalize for training stability

            # Layer 2: 256 → 256 neurons
            nn.Linear(hidden, hidden),
            nn.ReLU(),
            nn.LayerNorm(hidden),

            # Layer 3: 256 → 128 neurons
            nn.Linear(hidden, hidden // 2),
            nn.ReLU(),

            # Output: 128 → Q-value for each action
            nn.Linear(hidden // 2, n_actions)
        )

    def forward(self, x):
        """Pass observation through the network to get Q-values."""

```

```

        return self.network(x)

print("✅ Neural network architecture defined!")
# Quick sanity check
test_net = DQNNetwork(128, 5).to(DEVICE)
test_in = torch.randn(4, 128).to(DEVICE) # Batch of 4 observations
test_out = test_net(test_in)
print(f"    Network test: input shape {test_in.shape} → output shape {test_out.

```

✅ Neural network architecture defined!
 Network test: input shape torch.Size([4, 128]) → output shape torch.Size([4, 5]) ✅

```

In [21]: # -----
# CELL 5 : EXPERIENCE REPLAY MEMORY
# -----

class ReplayMemory:
    """
    Stores past experiences so the agent can learn from them later.

    Why? Learning from experiences in order creates correlations
    that destabilize training. Random sampling breaks those correlations.

    Think of it as the agent's 'notebook' of past folding attempts.
    """

    def __init__(self, capacity=50000):
        self.memory = collections.deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, done):
        """Save one experience."""
        self.memory.append((state, action, reward, next_state, done))

    def sample(self, batch_size):
        """Randomly sample a batch of experiences for learning."""
        batch = random.sample(self.memory, batch_size)
        states, actions, rewards, next_states, dones = zip(*batch)
        return (
            torch.FloatTensor(np.array(states)).to(DEVICE),
            torch.LongTensor(actions).to(DEVICE),
            torch.FloatTensor(rewards).to(DEVICE),
            torch.FloatTensor(np.array(next_states)).to(DEVICE),
            torch.FloatTensor(dones).to(DEVICE)
        )

    def __len__(self):
        return len(self.memory)

print("✅ Replay Memory defined!")

```

✅ Replay Memory defined!

In [22]:

```
# -----  
# CELL 6 : THE DOUBLE DQN AGENT  
# -----  
  
class DoubleDQNAgent:  
    """  
    Double DQN Agent – learns to fold proteins from 200,000 sequences.  
  
    'Double' means we use TWO neural networks:  
    1. Online Network : chooses actions, updated every step  
    2. Target Network : evaluates those actions, updated every N steps  
  
    This separation prevents the agent from chasing unstable targets  
    and makes learning much more stable.  
    """  
  
    def __init__(self, obs_size, n_actions=5,  
                  lr=0.0005, gamma=0.99,  
                  epsilon=1.0, epsilon_decay=0.9995,  
                  epsilon_min=0.05, batch_size=64,  
                  target_update_freq=500, memory_size=50000):  
  
        self.n_actions = n_actions  
        self.gamma = gamma  
        self.epsilon = epsilon  
        self.epsilon_decay = epsilon_decay  
        self.epsilon_min = epsilon_min  
        self.batch_size = batch_size  
        self.target_update_freq = target_update_freq  
        self.steps_done = 0  
  
        # — Two Networks -----  
        self.online_net = DQNNetwork(obs_size, n_actions).to(DEVICE)  
        self.target_net = DQNNetwork(obs_size, n_actions).to(DEVICE)  
        self.target_net.load_state_dict(self.online_net.state_dict())  
        self.target_net.eval() # Target net is never trained directly  
  
        # — Optimizer & Loss -----  
        self.optimizer = optim.Adam(self.online_net.parameters(), lr=lr)  
        self.loss_fn = nn.SmoothL1Loss() # Huber loss: stable for RL  
  
        # — Memory -----  
        self.memory = ReplayMemory(memory_size)  
  
        # — Metrics -----  
        self.loss_history = []  
  
    def choose_action(self, state):  
        """Epsilon-greedy: explore randomly OR pick best known action."""  
        if random.random() < self.epsilon:  
            return random.randint(0, self.n_actions - 1)  
  
        with torch.no_grad():
```

```

        obs_tensor = torch.FloatTensor(state).unsqueeze(0).to(DEVICE)
        q_values = self.online_net(obs_tensor)
        return q_values.argmax().item()

def store_experience(self, state, action, reward, next_state, done):
    """Save experience to replay memory."""
    self.memory.push(state, action, reward, next_state, float(done))

def learn(self):
    """
    Sample a random batch from memory and update the online network.

    Double DQN update:
    - Use online_net to CHOOSE the best next action
    - Use target_net to EVALUATE that action's value
    This prevents overestimation of Q-values.
    """
    if len(self.memory) < self.batch_size:
        return # Not enough experiences yet

    states, actions, rewards, next_states, dones = self.memory.sample(self.batch_size)

    # — Current Q-values (what we think now) —————
    current_q = self.online_net(states).gather(1, actions.unsqueeze(1)).squeeze(1)

    # — Target Q-values (what we should aim for) —————
    with torch.no_grad():
        # DOUBLE DQN: online net picks the action
        best_next_actions = self.online_net(next_states).argmax(1)
        # target net evaluates it
        next_q = self.target_net(next_states).gather(
            1, best_next_actions.unsqueeze(1)).squeeze(1)
        target_q = rewards + self.gamma * next_q * (1 - dones)

    # — Compute loss and backpropagate —————
    loss = self.loss_fn(current_q, target_q)
    self.optimizer.zero_grad()
    loss.backward()
    nn.utils.clip_grad_norm_(self.online_net.parameters(), 1.0) # Clip gradients
    self.optimizer.step()
    self.loss_history.append(loss.item())

    # — Periodically sync target network —————
    self.steps_done += 1
    if self.steps_done % self.target_update_freq == 0:
        self.target_net.load_state_dict(self.online_net.state_dict())

    # — Decay exploration —————
    self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay)

print("✅ Double DQN Agent defined!")

```

✅ Double DQN Agent defined!

In [23]:

```
# -----  
# CELL 7 : TRAINING LOOP  
# -----  
  
def train_ddqn(sequences, n_episodes=20000, save_interval=2000):  
    """  
    Main training loop for ISA-2.  
  
    For each episode:  
        1. Pick a random sequence from the dataset  
        2. Reset the environment with that sequence  
        3. Run until done (all beads placed or collision)  
        4. Agent learns from stored experiences  
    """  
  
    # Build agent with observation size from a sample env  
    sample_env = ProteinFoldingEnv3D(sequences[0])  
    obs_size = sample_env.obs_size  
    agent = DoubleDQNAgent(obs_size=obs_size, n_actions=5)  
  
    print(f"\n{'='*60}")  
    print(f"  ISA-2 – Double DQN Training")  
    print(f"  Dataset: {len(sequences)} sequences")  
    print(f"  Episodes: {n_episodes}")  
    print(f"  Observation size: {obs_size}")  
    print(f"  Device: {DEVICE}")  
    print(f"{'='*60}")  
  
    # — Metrics to track —————  
    ep_contacts = []  
    ep_rewards = []  
    rolling_contacts = []  
    best_contact = 0  
    best_positions = None  
    best_sequence = None  
  
    for episode in range(n_episodes):  
  
        # — Pick a random sequence each episode —————  
        seq = random.choice(sequences)  
        env = ProteinFoldingEnv3D(seq)  
        obs = env.reset()  
        done = False  
        total_reward = 0  
  
        # — Run one episode —————  
        while not done:  
            action = agent.choose_action(obs)  
            next_obs, reward, done, info = env.step(action)  
            agent.store_experience(obs, action, reward, next_obs, done)  
            agent.learn()  
            obs = next_obs  
            total_reward += reward
```

```

# — Track metrics —————
contacts = env.get_total_contacts()
ep_contacts.append(contacts)
ep_rewards.append(total_reward)

if contacts > best_contact:
    best_contact = contacts
    best_positions = list(env.positions)
    best_sequence = seq

window = min(200, len(ep_contacts))
rolling_contacts.append(np.mean(ep_contacts[-window:]))

# — Print progress —————
if (episode + 1) % 1000 == 0:
    print(f" Episode {episode+1:5d} | "
          f"Avg(200): {rolling_contacts[-1]:.2f} | "
          f"Best: {best_contact} | "
          f"ε: {agent.epsilon:.3f} | "
          f"Loss: {np.mean(agent.loss_history[-50:])} if agent.loss_hist
          f"Memory: {len(agent.memory)}")

print(f"\n✅ Training complete!")
print(f" Best contacts found: {best_contact}")
print(f" On sequence: {best_sequence}")

return agent, ep_contacts, rolling_contacts, best_positions, best_sequence

# — Start training —————
# Set n_episodes higher for better results (20000+ recommended)
# For a quick test, use 3000 episodes
agent, ep_contacts, rolling_avg, best_positions, best_seq = train_ddqn(
    train_seqs,
    n_episodes=5000
)

```

```
=====
ISA-2 – Double DQN Training
Dataset: 4500 sequences
Episodes: 5000
Observation size: 128
Device: cpu
=====
Episode 1000 | Avg(200): 2.47 | Best: 11 |  $\epsilon$ : 0.050 | Loss: 0.1074 | Memory: 23162
Episode 2000 | Avg(200): 2.83 | Best: 11 |  $\epsilon$ : 0.050 | Loss: 0.2145 | Memory: 48723
Episode 3000 | Avg(200): 2.90 | Best: 12 |  $\epsilon$ : 0.050 | Loss: 0.2581 | Memory: 50000
Episode 4000 | Avg(200): 3.16 | Best: 12 |  $\epsilon$ : 0.050 | Loss: 0.3252 | Memory: 50000
Episode 5000 | Avg(200): 3.04 | Best: 14 |  $\epsilon$ : 0.050 | Loss: 0.3970 | Memory: 50000

✓ Training complete!
Best contacts found: 14
On sequence: HHPHHHHHHHHPPHHPPHHHHPPHHHHPPPPHHHHHPHPPH
```

```
In [24]: # -----
# CELL 8 : TRAINING GRAPHS
# -----

def plot_ddqn_training(ep_contacts, rolling_avg, agent):
    """Comprehensive training analysis graphs for ISA-2."""

    fig, axes = plt.subplots(2, 2, figsize=(16, 12))
    fig.suptitle('ISA-2 – Double DQN Training Analysis\n(200,000 UniProt Sequences)',
                 fontsize=14, fontweight='bold')

    # — 1. Learning curve —
    ax = axes[0, 0]
    eps = range(len(ep_contacts))
    ax.plot(eps, ep_contacts, color='#a8d8ea', alpha=0.3, linewidth=0.5,
            label='Per-episode contacts')
    ax.plot(eps, rolling_avg, color='#e94560', linewidth=2.5,
            label='Rolling avg (200 ep)')
    ax.axhline(y=max(ep_contacts), color='#2ecc71', linestyle='--',
               linewidth=1.5, label=f'Best: {max(ep_contacts)}')
    ax.set_xlabel('Episode'); ax.set_ylabel('H-H Contacts')
    ax.set_title('Learning Curve', fontweight='bold')
    ax.legend(fontsize=9); ax.grid(True, alpha=0.3); ax.set_facecolor('#f8f9fa')

    # — 2. Loss curve —
    ax = axes[0, 1]
    if agent.loss_history:
        loss_smooth = np.convolve(agent.loss_history,
                                   np.ones(50)/50, mode='valid')
        ax.plot(agent.loss_history, color='#f39c12', alpha=0.2,
                linewidth=0.5, label='Raw loss')
        ax.plot(loss_smooth, color='#e74c3c', linewidth=2,
```

```

        label='Smoothed loss (50 step avg)')
ax.set_xlabel('Training Step'); ax.set_ylabel('Huber Loss')
ax.set_title('Network Loss Over Time', fontweight='bold')
ax.legend(fontsize=9); ax.grid(True, alpha=0.3); ax.set_facecolor('#f8f9fa')

# — 3. Exploration decay —————
ax = axes[1, 0]
n = len(ep_contacts)
epsilons = [max(0.05, 1.0 * (0.9995 ** i)) for i in range(n)]
ax.plot(range(n), epsilons, color='#9b59b6', linewidth=2)
ax.fill_between(range(n), epsilons, alpha=0.2, color='#9b59b6')
ax.set_xlabel('Episode')
ax.set_ylabel('Epsilon (Exploration Rate)')
ax.set_title('Exploration Rate Decay\n(1.0 = random, 0.05 = mostly learned',
            fontweight='bold')
ax.grid(True, alpha=0.3); ax.set_facecolor('#f8f9fa')

# — 4. Contacts distribution comparison —————
ax = axes[1, 1]
n_ep = len(ep_contacts)
first_half = ep_contacts[:n_ep//2]
second_half = ep_contacts[n_ep//2:]
max_val = max(ep_contacts) + 2
bins = range(0, max_val + 1)
ax.hist(first_half, bins=bins, color='#3498db', alpha=0.7,
        label=f'First 50% of training (avg={np.mean(first_half):.2f})',
        density=True)
ax.hist(second_half, bins=bins, color='#e74c3c', alpha=0.7,
        label=f'Second 50% of training (avg={np.mean(second_half):.2f})',
        density=True)
ax.set_xlabel('H-H Contacts per Episode')
ax.set_ylabel('Probability Density')
ax.set_title('Did the Agent Improve?\n(Red should shift right vs Blue)',
            fontweight='bold')
ax.legend(fontsize=9); ax.grid(True, alpha=0.3); ax.set_facecolor('#f8f9fa')

plt.tight_layout()
plt.savefig('isa2_training_analysis.png', dpi=150, bbox_inches='tight',
        facecolor='white')
plt.show()
print("📊 Training analysis saved as 'isa2_training_analysis.png'")

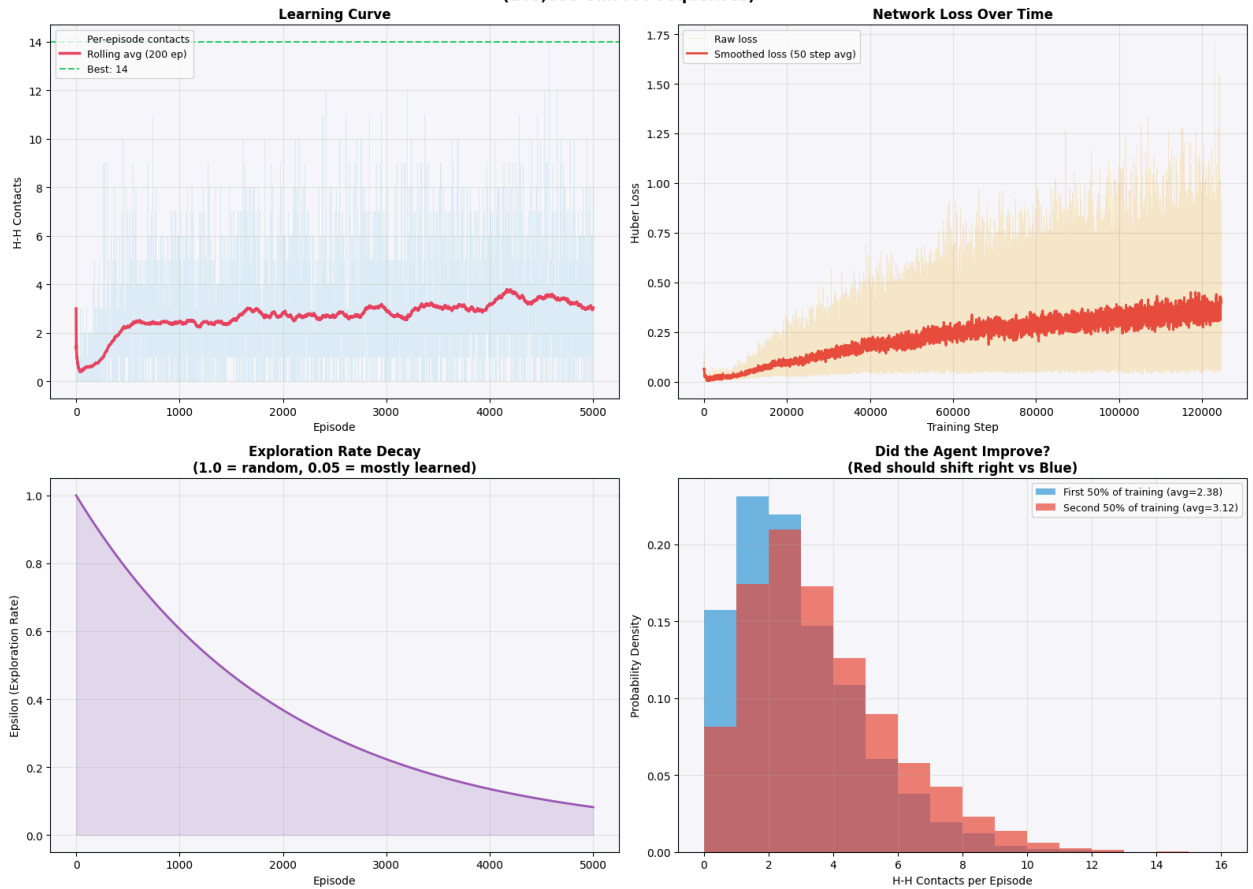
```

```

plot_ddqn_training(ep_contacts, rolling_avg, agent)

```

ISA-2 — Double DQN Training Analysis (200,000 UniProt Sequences)



Training analysis saved as 'isa2_training_analysis.png'

```
In [25]: # _____
# CELL 9 : 3D MATPLOTLIB VISUALIZATION
# _____

def visualize_3d_matplotlib(positions, sequence, title="3D Protein Fold"):
    """
    Static 3D visualization using Matplotlib.
    Shows the protein as a 3D chain with colored beads.
    """
    fig = plt.figure(figsize=(12, 10))
    ax = fig.add_subplot(111, projection='3d')

    xs = [p[0] for p in positions]
    ys = [p[1] for p in positions]
    zs = [p[2] for p in positions]

    # — Draw backbone —
    ax.plot(xs, ys, zs, color='#555555', linewidth=2.5,
            alpha=0.8, zorder=1)

    # — Draw H-H contact lines —
    pos_dict = {pos: i for i, pos in enumerate(positions)}
    contact_count = 0
    for i, pos in enumerate(positions):
```

```

    if sequence[i] != 'H': continue
    x, y, z = pos
    for dx,dy,dz in [(1,0,0),(-1,0,0),(0,1,0),(0,-1,0),(0,0,1),(0,0,-1)]:
        nb = (x+dx, y+dy, z+dz)
        if nb in pos_dict:
            j = pos_dict[nb]
            if j > i and abs(i-j) > 1 and sequence[j] == 'H':
                ax.plot([x,nb[0]], [y,nb[1]], [z,nb[2]],
                        color='#2ecc71', linewidth=3,
                        linestyle='--', alpha=0.9, zorder=2)
                contact_count += 1

# — Draw beads —————
for i, (x, y, z) in enumerate(positions):
    color = '#e74c3c' if sequence[i] == 'H' else '#3498db'
    size = 150 if sequence[i] == 'H' else 100
    ax.scatter(x, y, z, c=color, s=size, zorder=5,
              edgecolors='#2c3e50', linewidths=1.5, depthshade=True)

# — Mark start and end —————
ax.scatter(*positions[0], c='#f39c12', s=300, marker='*',
          zorder=6, label='Start')
ax.scatter(*positions[-1], c='#9b59b6', s=200, marker='D',
          zorder=6, label='End')

# — Labels —————
ax.set_xlabel('X axis', fontsize=10)
ax.set_ylabel('Y axis', fontsize=10)
ax.set_zlabel('Z axis', fontsize=10)
ax.set_title(f'{title}\nSequence: {sequence}\nH-H Contacts = {contact_count}',
            fontsize=11, fontweight='bold')

legend_elements = [
    mpatches.Patch(color='#e74c3c', label='H (Hydrophobic)'),
    mpatches.Patch(color='#3498db', label='P (Polar)'),
    mpatches.Patch(color='#2ecc71', label=f'H-H Contacts ({contact_count})')
]
ax.legend(handles=legend_elements, loc='upper left', fontsize=9)

plt.tight_layout()
plt.savefig('isa2_3d_fold_matplotlib.png', dpi=150, bbox_inches='tight',
          facecolor='white')
plt.show()
print(f"📸 3D matplotlib image saved as 'isa2_3d_fold_matplotlib.png'")
return contact_count

```

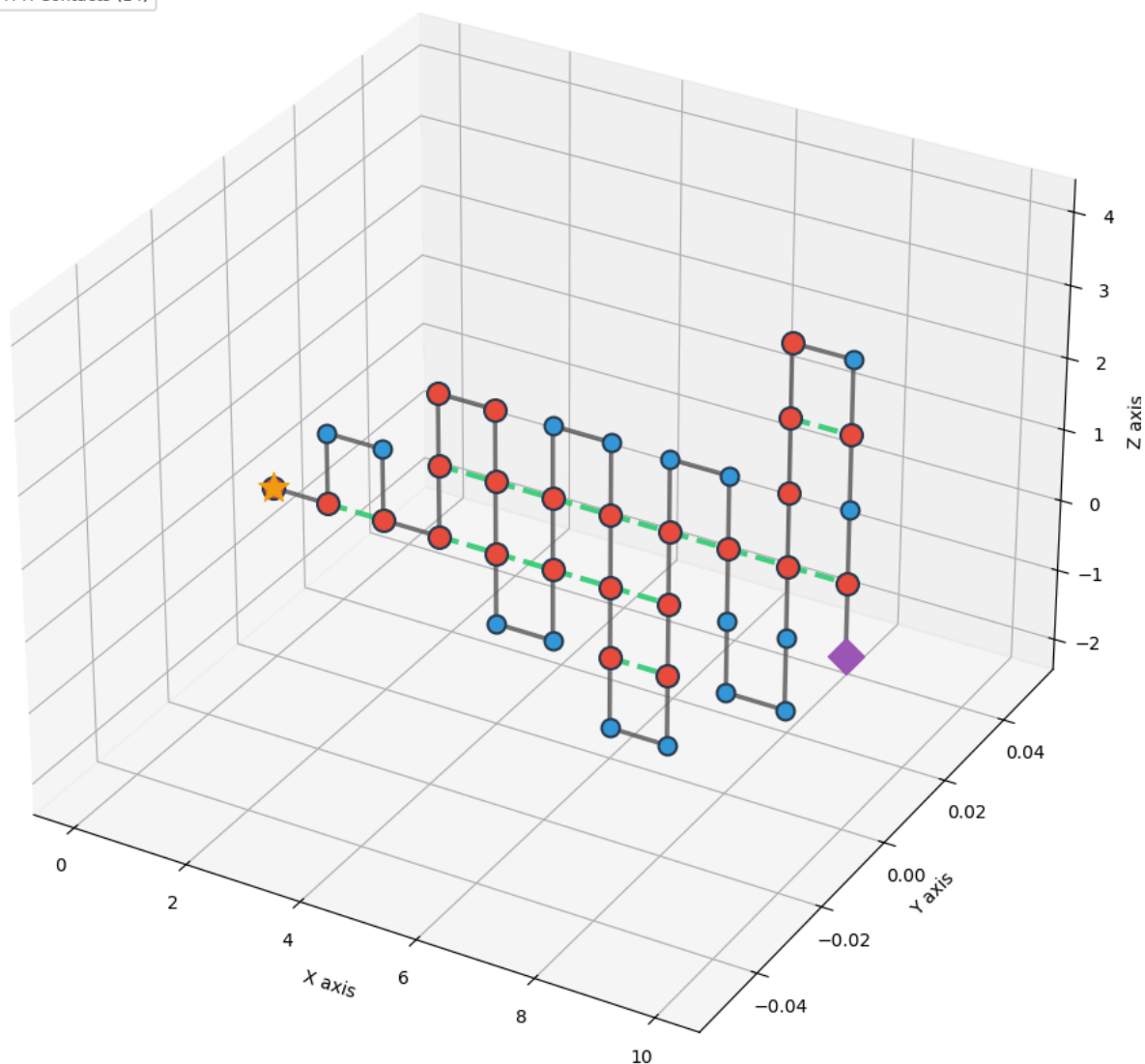
```

visualize_3d_matplotlib(best_positions, best_seq,
                        "Best 3D Protein Fold Found by DDQN Agent")

```

Best 3D Protein Fold Found by DDQN Agent
Sequence: HHPHHHHHHHHPHHPHHPHHPHHPHPPPHHHHPHHPH
H-H Contacts = 14

■ H (Hydrophobic)
■ P (Polar)
■ H-H Contacts (14)



📷 3D matplotlib image saved as 'isa2_3d_fold_matplotlib.png'

Out[25]: 14

```
In [26]: # _____
# CELL 10 : INTERACTIVE 3D PLOTLY VISUALIZATION
# _____

def visualize_3d_interactive(positions, sequence, title="Interactive 3D Protein Fold")
    """
    FULLY INTERACTIVE 3D visualization using Plotly.
    You can rotate, zoom, pan the protein in your browser!
    - Red   = Hydrophobic (H) bead
    - Blue  = Polar (P) bead
    - Green = H-H contact (reward-giving)
```

```

- Gray = Backbone chain
"""

traces = []
pos_dict = {pos: i for i, pos in enumerate(positions)}

# — Backbone trace —————
xs = [p[0] for p in positions]
ys = [p[1] for p in positions]
zs = [p[2] for p in positions]

traces.append(go.Scatter3d(
    x=xs, y=ys, z=zs,
    mode='lines',
    line=dict(color='#888888', width=4),
    name='Backbone chain',
    hoverinfo='none'
))

# — H beads —————
h_x = [positions[i][0] for i in range(len(positions)) if sequence[i]=='H']
h_y = [positions[i][1] for i in range(len(positions)) if sequence[i]=='H']
h_z = [positions[i][2] for i in range(len(positions)) if sequence[i]=='H']
h_labels = [f"Bead {i+1}: H (Hydrophobic)" for i in range(len(positions))]

traces.append(go.Scatter3d(
    x=h_x, y=h_y, z=h_z,
    mode='markers',
    marker=dict(size=10, color='#e74c3c',
        line=dict(color='#c0392b', width=2),
        opacity=0.9, symbol='circle'),
    name='H (Hydrophobic)',
    text=h_labels, hoverinfo='text'
))

# — P beads —————
p_x = [positions[i][0] for i in range(len(positions)) if sequence[i]=='P']
p_y = [positions[i][1] for i in range(len(positions)) if sequence[i]=='P']
p_z = [positions[i][2] for i in range(len(positions)) if sequence[i]=='P']
p_labels = [f"Bead {i+1}: P (Polar)" for i in range(len(positions)) if sec

traces.append(go.Scatter3d(
    x=p_x, y=p_y, z=p_z,
    mode='markers',
    marker=dict(size=7, color='#3498db',
        line=dict(color='#2980b9', width=1),
        opacity=0.8, symbol='circle'),
    name='P (Polar)',
    text=p_labels, hoverinfo='text'
))

# — H-H Contact lines —————
contact_count = 0

```

```

for i, pos in enumerate(positions):
    if sequence[i] != 'H': continue
    x, y, z = pos
    for dx, dy, dz in [(1,0,0), (-1,0,0), (0,1,0), (0,-1,0), (0,0,1), (0,0,-1)]:
        nb = (x+dx, y+dy, z+dz)
        if nb in pos_dict:
            j = pos_dict[nb]
            if j > i and abs(i-j) > 1 and sequence[j] == 'H':
                traces.append(go.Scatter3d(
                    x=[x, nb[0], None],
                    y=[y, nb[1], None],
                    z=[z, nb[2], None],
                    mode='lines',
                    line=dict(color='#2ecc71', width=6, dash='dash'),
                    name='H-H Contact' if contact_count == 0 else '',
                    showlegend=(contact_count == 0),
                    hoverinfo='none'
                ))
                contact_count += 1

# — Start/End markers —————
traces.append(go.Scatter3d(
    x=[positions[0][0]], y=[positions[0][1]], z=[positions[0][2]],
    mode='markers',
    marker=dict(size=15, color='#f39c12', symbol='diamond',
        line=dict(color='black', width=2)),
    name='Start (N-terminus)'
))
traces.append(go.Scatter3d(
    x=[positions[-1][0]], y=[positions[-1][1]], z=[positions[-1][2]],
    mode='markers',
    marker=dict(size=12, color='#9b59b6', symbol='square',
        line=dict(color='black', width=2)),
    name='End (C-terminus)'
))

# — Bead number labels (every 5th bead) —————
label_indices = list(range(0, len(positions), max(1, len(positions)//10)))
traces.append(go.Scatter3d(
    x=[positions[i][0] for i in label_indices],
    y=[positions[i][1] for i in label_indices],
    z=[positions[i][2] for i in label_indices],
    mode='text',
    text=[str(i+1) for i in label_indices],
    textfont=dict(size=10, color='black'),
    name='Bead numbers',
    hoverinfo='none'
))

# — Layout —————
fig = go.Figure(data=traces)
fig.update_layout(
    title=dict(

```

```
text=f"<b>{title}</b><br>"
    f"<sub>Sequence: {sequence[:30]}{'...' if len(sequence)>30 else ''}"
    f"<sub>H-H Contacts (Reward) = {contact_count} | "
    f"Length = {len(sequence)} beads</sub>",
font=dict(size=14)
),
scene=dict(
    xaxis=dict(title='X', gridcolor='lightgray', showbackground=True,
                backgroundcolor='rgba(240,244,248,0.5)'),
    yaxis=dict(title='Y', gridcolor='lightgray', showbackground=True,
                backgroundcolor='rgba(240,244,248,0.5)'),
    zaxis=dict(title='Z', gridcolor='lightgray', showbackground=True,
                backgroundcolor='rgba(240,244,248,0.5)'),
    camera=dict(eye=dict(x=1.5, y=1.5, z=1.2))
),
paper_bgcolor='white',
plot_bgcolor='white',
legend=dict(x=0.01, y=0.99, bgcolor='rgba(255,255,255,0.9)',
            bordercolor='#cccccc', borderwidth=1),
width=900, height=700,
annotations=[
    dict(
        text=(f"🔴 Red = Hydrophobic (H) bead – wants to hide from water  

               🔵 Blue = Polar (P) bead – happy on the surface<br>"
              f"<span style='color:green'>-- Green = H-H Contact (+1 reward)  

               🖱️ Rotate: drag | Zoom: scroll | Pan: right-click drag"),
        xref='paper', yref='paper', x=0.01, y=0.15,
        showarrow=False, align='left',
        bgcolor='rgba(255,255,255,0.85)',
        bordercolor='#cccccc', borderwidth=1,
        font=dict(size=11)
    )
]
)
```

Save as HTML (fully interactive in browser)

```
fig.write_html('isa2_3d_interactive.html')
fig.show()
```

```
print(f"\n🌐 Interactive 3D saved as 'isa2_3d_interactive.html'")
print(f"   Open this file in any browser to rotate/zoom the protein!")
print(f"   Total H-H contacts: {contact_count}")
```

return contact_count

```
c = visualize_3d_interactive(best_positions, best_seq,
```

"DDQN Agent – Best 3D Protein Fold")

🌐 Interactive 3D saved as 'isa2_3d_interactive.html'
Open this file in any browser to rotate/zoom the protein!
Total H-H contacts: 14

```
In [27]: # _____  
# CELL 11 : 3D ANIMATED FOLDING (step-by-step plotly)  
# _____  
  
def animate_folding_3d(positions, sequence, max_frames=None):  
    """  
    ANIMATED 3D visualization – watch the protein build itself step by step.  
    A slider lets you scrub through each placement step.
```

```

"""
n = len(positions)
if max_frames is not None:
    n = min(n, max_frames)

frames = []
for step in range(2, n + 1):
    pos_subset = positions[:step]
    seq_subset = sequence[:step]

    xs = [p[0] for p in pos_subset]
    ys = [p[1] for p in pos_subset]
    zs = [p[2] for p in pos_subset]

    h_x = [pos_subset[i][0] for i in range(step) if seq_subset[i]=='H']
    h_y = [pos_subset[i][1] for i in range(step) if seq_subset[i]=='H']
    h_z = [pos_subset[i][2] for i in range(step) if seq_subset[i]=='H']

    p_x = [pos_subset[i][0] for i in range(step) if seq_subset[i]=='P']
    p_y = [pos_subset[i][1] for i in range(step) if seq_subset[i]=='P']
    p_z = [pos_subset[i][2] for i in range(step) if seq_subset[i]=='P']

    # Count contacts at this step
    pd = {pos_subset[i]: i for i in range(step)}
    contacts = 0
    for i, pos in enumerate(pos_subset):
        if seq_subset[i] != 'H': continue
        x,y,z = pos
        for dx,dy,dz in [(1,0,0),(-1,0,0),(0,1,0),(0,-1,0),(0,0,1),(0,0,-1)]:
            nb = (x+dx,y+dy,z+dz)
            if nb in pd:
                j = pd[nb]
                if j > i and abs(i-j) > 1 and seq_subset[j] == 'H':
                    contacts += 1

    frame_data = [
        go.Scatter3d(x=xs, y=ys, z=zs, mode='lines',
                     line=dict(color='#888888', width=3), name='Chain'),
        go.Scatter3d(x=h_x, y=h_y, z=h_z, mode='markers',
                     marker=dict(size=10, color='#e74c3c', opacity=0.9),
                     name='H beads'),
        go.Scatter3d(x=p_x, y=p_y, z=p_z, mode='markers',
                     marker=dict(size=7, color='#3498db', opacity=0.8),
                     name='P beads'),
        # Newest bead highlighted
        go.Scatter3d(x=[pos_subset[-1][0]], y=[pos_subset[-1][1]],
                     z=[pos_subset[-1][2]], mode='markers',
                     marker=dict(size=14, color='#f39c12',
                                line=dict(color='black',width=2)),
                     name=f'Current: {seq_subset[-1]}'),
    ]
    frames.append(go.Frame(
        data=frame_data,

```

```

        name=str(step),
        layout=go.Layout(title_text=(
            f"Step {step}/{n}: Placing bead {step} ({sequence[step-1]}) |
            f"H-H Contacts so far: {contacts}"
        ))
    ))

# Initial frame
fig = go.Figure(
    data=frames[0].data,
    frames=frames,
    layout=go.Layout(
        title=f"Animated 3D Protein Folding – Watch Each Step",
        scene=dict(
            xaxis=dict(title='X'), yaxis=dict(title='Y'), zaxis=dict(title=
        ),
        paper_bgcolor='white',
        width=900, height=650,
        sliders=[{
            'active': 0,
            'steps': [{ 'method': 'animate',
                        'args': [[f.name], { 'mode': 'immediate',
                                                'frame': { 'duration': 300},
                                                'transition': { 'duration': 200},
                                                'label': f.name} for f in frames],
                        'x': 0.1, 'len': 0.8, 'y': 0,
                        'currentvalue': { 'prefix': 'Step: ', 'font': { 'size': 14}},
                    }],
            updatemenus=[{
                'buttons': [
                    { 'args': [None, { 'frame': { 'duration': 400, 'redraw': True,
                                                'fromcurrent': True}],
                        'label': '▶ Play',
                        'method': 'animate'},
                    { 'args': [[None], { 'frame': { 'duration': 0},
                                                'mode': 'immediate'}],
                        'label': '⏸ Pause',
                        'method': 'animate'}
                ],
                'type': 'buttons',
                'x': 0.1, 'y': 1.05
            }
        ]
    )
)
fig.write_html('isa2_3d_animated.html')
fig.show()
print("🎬 Animated 3D fold saved as 'isa2_3d_animated.html'")
print("    Press Play to watch the protein fold step by step!")

```

animate_folding_3d(best_positions, best_seq)

 Animated 3D fold saved as 'isa2_3d_animated.html'
Press Play to watch the protein fold step by step!

```
In [28]: # _____  
# CELL 12 : VALIDATION – Test on sequences never seen in training  
# _____  
  
def validate_agent(agent, val_sequences, n_test=200):  
    """  
    Test the trained agent on sequences it has NEVER seen.  
    This measures GENERALIZATION – the real test of learning.  
    """  
    print(f"\n{'='*50}")  
    print(f"  Validating on {n_test} held-out sequences...")  
    print(f"{'='*50}")
```

```

results = []
agent_contacts = []
random_contacts = []

test_seqs = random.sample(val_sequences, min(n_test, len(val_sequences)))

for seq in test_seqs:
    # — Agent's fold —————
    env = ProteinFoldingEnv3D(seq)
    obs = env.reset()
    done = False
    while not done:
        with torch.no_grad():
            obs_t = torch.FloatTensor(obs).unsqueeze(0).to(DEVICE)
            action = agent.online_net(obs_t).argmax().item()
            obs, _, done, _ = env.step(action)
        agent_c = env.get_total_contacts()

    # — Random fold (baseline) —————
    env.reset()
    done = False
    while not done:
        _, _, done, _ = env.step(random.randint(0, 4))
    rand_c = env.get_total_contacts()

    agent_contacts.append(agent_c)
    random_contacts.append(rand_c)

agent_avg = np.mean(agent_contacts)
random_avg = np.mean(random_contacts)
improvement = ((agent_avg - random_avg) / max(random_avg, 0.001)) * 100

print(f"\n 📊 Validation Results ({n_test} sequences):")
print(f"     Agent average contacts : {agent_avg:.3f}")
print(f"     Random average contacts: {random_avg:.3f}")
print(f"     Improvement over random: {improvement:.1f}%")

# — Comparison scatter plot —————
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('ISA-2 Validation: Agent vs Random Baseline',
             fontsize=13, fontweight='bold')

axes[0].scatter(random_contacts, agent_contacts, alpha=0.5,
                color='#3498db', s=40, edgecolors='none')
lim = max(max(random_contacts), max(agent_contacts)) + 1
axes[0].plot([0, lim], [0, lim], 'r--', label='Equal performance line')
axes[0].set_xlabel('Random Agent Contacts')
axes[0].set_ylabel('DDQN Agent Contacts')
axes[0].set_title('Agent vs Random\n(Above red line = agent is better)')
axes[0].legend(); axes[0].grid(True, alpha=0.3)
axes[0].set_facecolor('#f8f9fa')

```

```

axes[1].hist(random_contacts, bins=15, color='#e74c3c', alpha=0.7,
              label=f'Random (avg={random_avg:.2f})', density=True)
axes[1].hist(agent_contacts, bins=15, color='#2ecc71', alpha=0.7,
              label=f'DDQN Agent (avg={agent_avg:.2f})', density=True)
axes[1].axvline(x=random_avg, color='#e74c3c', linewidth=2, linestyle='--')
axes[1].axvline(x=agent_avg, color='#2ecc71', linewidth=2, linestyle='--')
axes[1].set_xlabel('H-H Contacts')
axes[1].set_ylabel('Probability Density')
axes[1].set_title(f'Contacts Distribution\nImprovement: {improvement:.1f}%')
axes[1].legend(fontsize=10); axes[1].grid(True, alpha=0.3)
axes[1].set_facecolor('#f8f9fa')

plt.tight_layout()
plt.savefig('isa2_validation.png', dpi=150, bbox_inches='tight',
            facecolor='white')

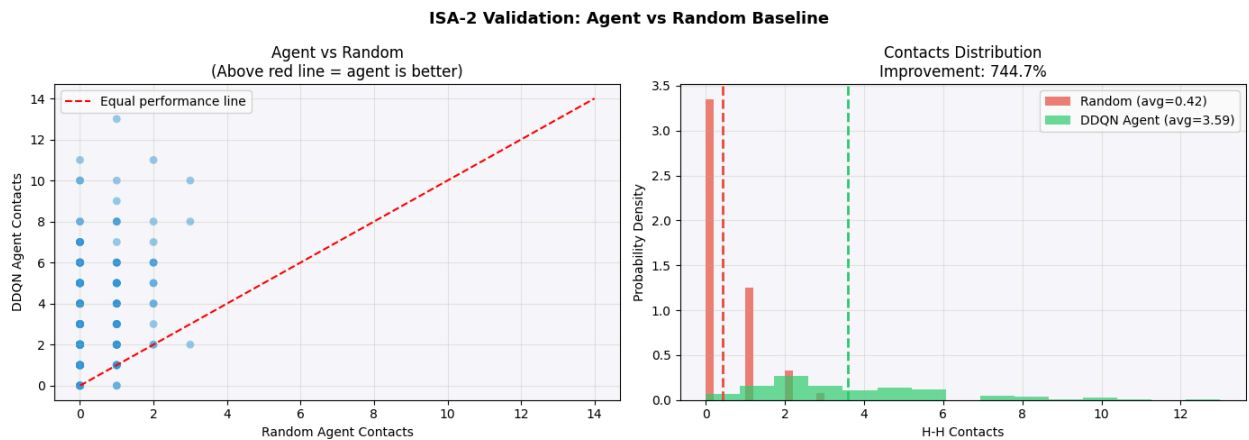
plt.show()
print("📊 Validation plot saved as 'isa2_validation.png'")
return agent_avg, random_avg, improvement

```

```
validate_agent(agent, val_seqs, n_test=200)
```

```
=====
Validating on 200 held-out sequences...
=====
```

📊 Validation Results (200 sequences):
 Agent average contacts : 3.590
 Random average contacts: 0.425
 Improvement over random: 744.7%



📊 Validation plot saved as 'isa2_validation.png'

```
Out[28]: (np.float64(3.59), np.float64(0.425), np.float64(744.7058823529412))
```

```

In [29]: # -----
# CELL 13 : FINAL SUMMARY
# -----

print("\n" + "="*60)
print("  ISA-2 COMPLETE – All outputs:")
print("="*60)

```

```

print(" 📊 isa2_training_analysis.png    – 4-panel training curves")
print(" 📷 isa2_3d_fold_matplotlib.png    – Static 3D image")
print(" 🌐 isa2_3d_interactive.html        – Interactive 3D (rotate/zoom)")
print(" 🎬 isa2_3d_animated.html           – Step-by-step animated 3D fold")
print(" 📊 isa2_validation.png              – Generalization test results")
print("=="*60)
print("""
HOW TO VIEW THE INTERACTIVE FILES:
  1. In Colab: Files panel (left) → download .html files
  2. Open in Chrome/Firefox – full 3D rotation, zoom, pan
  3. Or run in Colab directly – the cell output shows it inline

WHAT TO LOOK FOR:
  • Red beads clustering together = agent learned hydrophobic core
  • Green dashed lines = H-H contacts (reward earned)
  • More contacts = better fold = lower Gibbs Free Energy
""")

```

=====

ISA-2 COMPLETE – All outputs:

=====

```

📊 isa2_training_analysis.png    – 4-panel training curves
📷 isa2_3d_fold_matplotlib.png    – Static 3D image
🌐 isa2_3d_interactive.html        – Interactive 3D (rotate/zoom)
🎬 isa2_3d_animated.html           – Step-by-step animated 3D fold
📊 isa2_validation.png              – Generalization test results

```

=====

HOW TO VIEW THE INTERACTIVE FILES:

1. In Colab: Files panel (left) → download .html files
2. Open in Chrome/Firefox – full 3D rotation, zoom, pan
3. Or run in Colab directly – the cell output shows it inline

WHAT TO LOOK FOR:

- Red beads clustering together = agent learned hydrophobic core
- Green dashed lines = H-H contacts (reward earned)
- More contacts = better fold = lower Gibbs Free Energy